Reviewer Pack — Minimal Root Count of Integer Polynomials and Resolution Pro…

Entry 00c9cd4749eb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 09:59:52 UTC

Minimal Root Count of Integer Polynomials and Resolution Proof Width

Entry ID: `00c9cd4749eb` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-03 09:59:52 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Root Theory) **Field B** (complexity object): Resolution Proof Complexity

Statement:

> For any CNF φ with m clauses, the minimal number of distinct roots of its associated integer polynomial p(φ) is at most a constant factor away from the resolution proof width w(φ), such that |#roots(p(φ))| ≤ c * w(φ), where c is an absolute constant.

Rationale (proposer's reasoning):

> Root theory provides a way to study the algebraic structure of polynomials, which could reveal underlying patterns in the computation of resolutions. If the minimal root count of integer polynomials correlates with resolution proof width, it may offer new insights into the complexity of SAT solving and provide alternative approaches to lower-bound analysis.

Taxonomy category: `algebraic_geometry` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `bd97c64df504b3a4`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a CNF φ, if |#roots(p(φ))| ≤ 1.5 * w(φ) where c=1.5 is an absolute constant and #roots(p(φ)) and w(φ) are calculated for the same φ.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:Minimal Root Count Integer Polynomials AND Resolution Proof Width - algebraic geometry AND integer polynomial roots AND resolution proof complexity - root theory IN algebraic geometry AND resolution proof width IN computational complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def polynomial(cnf):
        n = len(cnf)
        x = [Fraction(1, i) if i > 0 else -Fraction(1, -i) for i in range(-n, n+1)]
        p = sum([sum([x[i] ** abs(lit) for lit in clause]) for clause in cnf])
        return p

    def resolution_width(cnf):
        clauses = set(tuple(sorted(clause)) for clause in cnf)
        resolved = set()
        while True:
            new_resolved = set()
            for clause1, clause2 in itertools.combinations(resolved, 2):
                if len(set(clause1) & set(clause2)) == 1:
                    new_clause = [lit for lit in clause1 + clause2 if lit not in (set(clause1) & s
                    new_resolved.add(tuple(sorted(new_clause)))
            if new_resolved.issubset(resolved):
                break
            resolved.update(new_resolved)
        return len(resolved)

    def count_distinct_roots(p):
        roots = set()
        for i in range(-100, 101):
            val = p.subs(x[i])
            if abs(val) < 1e-6:
                roots.add(i)
        return len(roots)
```

```
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        cnf = [[random.choice([-i, i]) for _ in range(random.randint(1, 3))] for _ in range(n)]
        p = polynomial(cnf)
        w = resolution_width(cnf)
        roots_count = count_distinct_roots(p)

        results.append({
            "n": n,
            "roots_count": roots_count,
            "w": w
        })

    mean_roots_count = sum(result["roots_count"] for result in results) / len(results)
    mean_w = sum(result["w"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if abs(result["roots_count"] - 1.5 * result["w"

    return {
        "metric_name": "Root Count vs Resolution Width",
        "metric_value": mean_roots_count,
        "instances_tested": len(results),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": support_fraction >= 0.8,
        "counterexample": "" if support_fraction >= 0.8 else f"mean_roots_count={mean_roots_count}
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"mean_roots_count does not satisfy the conjectur
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d674a89f.py", line 83, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d674a89f.py", line 53, in run_trial
    cnf = [[random.choice([-i, i]) for _ in range(random.randint(1, 3))] for _ in range(n)]
                                ^
NameError: name 'i' is not defined. Did you mean: 'id'?
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed due to an undefined variable ‘i’, which prevented the computation from completing and producing data. | next: Review the test code for the error and correct it before re-running the test.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13915
2	preregistration	ollama_remote	glm4:latest	0	0	9493
3	novelty	ollama_remote	glm4:latest	0	0	8262
4	novelty	ollama_remote	glm4:latest	0	0	10146
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15605
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9905
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15081
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10724
9	judge	ollama_remote	glm4:latest	0	0	13899

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 107030 ms total latency. Provider mix: {‘ollama_remote’: 9}

(full prompt+response transcripts available in `research/audit/00c9cd4749eb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/00c9cd4749eb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/00c9cd4749eb.tar.gz` (if generated)